

NEYO

NEYO Founder Manual

A non-coder friendly operating guide for running, testing,
deploying and selling NEYO.

Version: 2026-06-24

Audience: Founder / non-coder operator

Company: NEYO

Scope: Company operations, School OS testing, local setup, deployment, pricing, marketing, and safe growth.

Page 1 — What NEYO is

NEYO is a company that builds operating systems for organizations.

Today the strongest live product is **NEYO School OS**. The wider company roadmap also includes **Business OS**, **Farm OS**, and **Creator OS**. The product must be operated from inside NEYO itself through **NEYO Ops**.

The founder should think of NEYO as:

- one company;
- many operating systems;
- one shared platform layer;
- one internal company cockpit;
- many customer-facing products.

The most important rule is simple:

NEYO must run NEYO inside NEYO.

Page 2 — The founder's daily control room

Your main company control room is:

NEYO Ops → Business Operations

This is where you manage:

- company documents;
- pricing catalog;
- school accounts;
- subscriptions;
- grace-period enforcement;
- central billing;
- contracts;
- customer communication;
- YouTube posting calendar;
- brand assets;
- OS lifecycle planning;
- landing page content.

If something is company-owned, it should live in NEYO Ops, not inside a school settings page.

Page 3 — Product map

Current NEYO OS map:

OS	Status	Purpose
School OS	Live	Schools, fees, attendance, exams, staff, learning, parent communication
Business OS	Waitlist / planned	SMEs, stock, customers, sales, teams
Farm OS	Waitlist / planned	Cooperatives, farm records, inventory, teams
Creator OS	Waitlist / planned	Creator businesses, content, sales, clients

All OSes share the platform layer:

- login;
 - tenancy;
 - billing;
 - audit logs;
 - notifications;
 - storage;
 - search;
 - calendar;
 - NEYO Ops support.
-

Page 4 — Important links inside the app

Common founder paths:

/founder	NEYO Ops
/settings/billing	school billing page
/settings/payments	school fee payment credentials
/settings/school	school profile and branding
/learning-videos	YouTube learning module
/privacy	live Privacy Policy
/terms	live Terms of Service
/os/school/login	School OS login
/os/business/login	Business OS login route
/os/farm/login	Farm OS login route
/os/creator/login	Creator OS login route

Public website:

/	NEYO company landing page
---	---------------------------

Page 5 — What must never be exposed publicly

The public landing page and marketing copy should show **features and outcomes only**.

Never expose:

- API keys;
- credentials;
- database internals;
- provider passkeys;
- private implementation details;
- internal prompts;
- Daraja secrets;
- cloud secret names;
- exact infrastructure secrets;
- any private school data.

Public copy should say:

"Secure school payments and receipts."

Not:

"We use this provider secret, this callback token, this table, and this database field."

Page 6 — Local testing tonight: simple version

To test NEYO locally on a laptop:

```
cd /home/user/neyo/neyo
npm install
./node_modules/.bin/prisma generate
./node_modules/.bin/prisma migrate deploy
npm run db:seed
npm run dev
```

Open:

```
http://localhost:3000
```

Founder login commonly seeded in dev:

```
support@neyo.co.ke
Karibu2026!
```

If the dev server is already running, refresh the browser.

Page 7 — Local testing: safest command order

When node_modules disappear in the sandbox, run:

```
npm install
```

Then:

```
./node_modules/.bin/prisma generate  
./node_modules/.bin/prisma migrate deploy  
npm run db:seed  
NODE_OPTIONS=--max-old-space-size=1536 npm run typecheck  
npm run test:roles
```

Use this for dev server in the sandbox:

```
(setsid npm run dev </dev/null >/tmp/neyo-dev.log 2>&1 &) ; sleep 18
```

Check log:

```
tail -80 /tmp/neyo-dev.log
```

Page 8 — Role safety

NEYO has 16 canonical roles. The role test must pass:

```
npm run test:roles
```

Expected:

```
24 passed, 0 failed
```

This confirms:

- roles exist;
- permission matrix is complete;
- tenant isolation tests pass;
- cross-tenant reads are blocked;
- ID generation works.

Never ship major changes if role tests fail.

Page 9 — How to test NEYO Ops

1. Login as SUPER_ADMIN.

2. Open:

/founder

3. Click **Business Operations**.

4. Check these sections:

- NEYO Business OS Cockpit;
- OS Lifecycle Board;
- Pricing & Package Editor;
- YouTube Management & Posting Hub;
- Contract Signing Management;
- Customer ↔ NEYO Communication Hub;
- Subscriptions & Billing Override;
- Grace-period enforcement;
- Brand editor;
- Landing Page Content Editor.

Each save should show a toast and write an audit log.

Page 10 — Pricing management

Pricing is edited in:

NEYO Ops → Business Operations → Pricing & Package Editor

The live catalog is stored as:

PlatformSetting: neyo_pricing_catalog

Rules:

- new subscriptions use the current catalog;
- existing subscriptions keep `grandfatheredPrice`;
- SMS is not inside packages;
- SMS is sold as an out-of-package top-up;
- price edits are SUPER_ADMIN-only;
- changes are audit logged.

Test:

```
./node_modules/.bin/tsx scripts/i48-pricing-catalog-test.ts
```

Page 11 — Central NEYO money

School fees and NEYO subscription money are separate.

School fees:

```
Settings → Payments  
PaymentCredential  
Payment  
Invoice
```

NEYO company subscription money:

```
SubscriptionPayment  
/api/billing/public-stk  
/api/billing/central-callback
```

Central account references look like:

```
NEYO-<school-slug>
```

When paid, the callback reconnects the school automatically.

Page 12 — Expired account reconnect

When a school is suspended, the root layout shows the expired checkout screen.

The school enters a phone number and receives an M-Pesa STK prompt.

Flow:

Suspended school → enter phone → central STK → callback → subscription ACTIVE → page reload → dashboard

Test:

```
./node_modules/.bin/tsx scripts/i49-central-money-reconnect-test.ts
```

Page 13 — Grace-period enforcement

Daily job:

```
subscription-state-machine
```

Schedule:

```
Daily 01:00 EAT
```

Flow:

```
ACTIVE overdue → GRACE + notice  
GRACE ending soon → warning  
GRACE expired → final notice → SUSPENDED
```

Data rule:

Never delete school data because of non-payment.

Test:

```
./node_modules/.bin/tsx scripts/i48-grace-enforcement-test.ts
```

Page 14 — Company documents

Editable in:

```
NEYO Ops → Business Operations → Legal editor
```

Settings:

```
privacy_policy  
terms_of_service
```

Public pages:

```
/privacy  
/terms
```

This means you can update legal copy without touching code.

Test:

```
./node_modules/.bin/tsx scripts/i48-company-documents-test.ts
```

Page 15 — Contracts

Contracts are managed in:

NEYO Ops → Contract Signing Management

Model:

NeyoContract

Public signing path:

/contracts/sign/[token]

A signer types:

- name;
- role;
- typed signature;
- agreement confirmation.

Audit action:

platform.contract_signed

Test:

./node_modules/.bin/tsx scripts/i48-contract-signing-test.ts

Page 16 — Customer communication hub

Schools can contact NEYO from:

Settings → Billing → Contact NEYO

NEYO receives messages in:

NEYO Ops → Customer ↔ NEYO Communication Hub

Models:

NeyoCustomerThread
NeyoCustomerMessage

Use it for:

- billing issues;
- account issues;
- onboarding questions;
- support records;
- renewal communication.

Test:

```
./node_modules/.bin/tsx scripts/i48-customer-neyo-hub-test.ts
```

Page 17 — Brand assets

Brand assets are edited in:

NEYO Ops → NEYO Global Branding & Asset Editor

Editable:

- logo;
- colors;
- favicon;
- PWA icon;
- Apple touch icon;
- wordmarks;
- Bundi mascot images;
- pattern tile.

Root metadata reads these settings.

Test:

```
./node_modules/.bin/tsx scripts/i48-brand-assets-test.ts
```

Page 18 — Landing page content

Landing content is edited in:

NEYO Ops → Landing Page Content Editor

Stored as:

PlatformSetting: neyo_landing_content

Content includes:

- hero copy;
- CTAs;
- nav;
- product cards;
- trust stats;
- media slots;
- footer links;
- SEO title;
- SEO description;
- Open Graph image.

Public page consumes it at:

/

Page 19 — Landing page design direction

The landing page should borrow the **clarity** of Odoo, not copy Odoo.

Good direction:

- simple;
- product-first;
- warm white background;
- app grid;
- clear CTAs;
- real screenshots;
- no hype;
- no generic startup language.

Avoid:

- purple glow;
 - fake charts;
 - over-dark hero;
 - stock photos;
 - fake testimonials;
 - internal secrets;
 - exaggerated claims.
-

Page 20 — YouTube learning

Path:

/learning-videos

Works in two modes:

1. Saved videos search — works without YouTube API key.
2. Live YouTube search — works when `YOUTUBE\_API\_KEY` is configured.

Search uses:

- strict safe search;
- embeddable videos only;
- education category;
- Kenya region;
- English relevance.

Videos play inside NEYO using privacy-enhanced embeds.

Page 21 — YouTube ads reality

NEYO can reduce distractions by:

- embedding inside NEYO;
- hiding comments;
- avoiding external YouTube browsing;
- using privacy-enhanced embed URLs;
- showing recommended learning ideas;
- not exposing downloads.

But NEYO cannot fully guarantee zero YouTube ads for third-party YouTube videos.

True zero-ad options:

- school-owned hosted videos;
 - uploaded school-owned content;
 - YouTube-side entitlement;
 - future premium controlled media pipeline.
-

Page 22 — Class casting

Learning videos can be cast from teacher phone to class screen.

Flow:

Teacher saves video → Cast to class screen → NEYO creates cast code → TV/projector opens `/learning-videos/cast/[code]`

Students can later see videos shown in class.

Test:

```
./node_modules/.bin/tsx scripts/i27-youtube-learning-test.ts
```

Page 23 — Hardware rule

Hardware must never show connected unless truly connected.

Allowed connection types:

- USB;
- Serial;
- Bluetooth;
- Wi-Fi/LAN;
- Keyboard/HID input;
- tracker feed for GPS.

A test tool must never fake connection status.

Test:

```
./node_modules/.bin/tsx scripts/i47-hardware-deferred-seams-test.ts
```

Page 24 — School profile and branding

School-owned branding lives in:

Settings → School

This is separate from NEYO company branding.

School branding controls:

- school logo;
- school colors;
- motto;
- public school profile;
- document branding;
- receipt/report styling.

NEYO company branding lives in NEYO Ops.

Page 25 — Payments for schools

School fee collection credentials live in:

Settings → Payments

These are tenant-owned.

Use them for:

- parent fee payments;
- invoice callbacks;
- receipts;
- finance ledger;
- school Paybill/Till.

Do not put NEYO company subscription credentials here.

See:

`docs/PAYMENTS-DEVELOPER-GUIDE.md`

Page 26 — Finance testing checklist

To test finance:

1. Open Finance.
2. Create or find invoice.
3. Trigger STK or record payment.
4. Confirm invoice balance changes.
5. Confirm receipt PDF works.
6. Confirm SMS/notification path if configured.
7. Confirm audit log exists.
8. Confirm role restrictions:
 - Bursar can record payments.
 - Teacher cannot manage finance.

Run role tests after finance changes.

Page 27 — Student testing checklist

Test:

- create student;
- edit student;
- guardian link;
- parent login;
- student ID card;
- bulk ID cards;
- transfer letter;
- leaving certificate vault;
- class/stream filters;
- document branding;
- tenant isolation.

Screenshots should show real Kenyan school context, not placeholder names.

Page 28 — Attendance testing checklist

Test:

- class attendance;
- absent SMS path;
- hostel attendance;
- staff attendance;
- GPS/geofence if configured;
- duplicate prevention;
- teacher permissions;
- principal summary.

Never let hardware attendance say connected unless a real device/feed exists.

Page 29 — Academics testing checklist

Test:

- departments;
- subjects;
- HOD assignment;
- co-curricular department;
- timetable;
- marks entry;
- report cards;
- CBC reports;
- exam materials;
- duty roster;
- print outputs.

For timetable printing, the output should fill A4 correctly and not behave like a compressed screenshot.

Page 30 — Learning testing checklist

Test:

- LMS notes;
- homework;
- quizzes;
- forum;
- YouTube learning search;
- saved videos;
- shown-in-class videos;
- class screen casting;
- online class entry;
- mobile layout.

No download action should appear for YouTube videos.

Page 31 — Staff and HR testing checklist

Test:

- staff creation;
- staff import;
- TSC/KRA/National ID fields;
- contract type;
- contract end date;
- staff attendance;
- payroll;
- payslips;
- role changes;
- staff permissions.

Staff contracts exist at staff-profile level; deeper templates can be added later if checklist asks.

Page 32 — Hostel testing checklist

Test:

- hostel creation;
 - room creation;
 - bed allocation;
 - mixed hostel rules;
 - transfer;
 - curfew attendance;
 - missing-boarder SMS;
 - freed bed tracking;
 - boarding/day school gating.
-

Page 33 — Library testing checklist

Test:

- book creation;
 - issue book;
 - return book;
 - late fine amount;
 - camera barcode scanner;
 - external barcode scanner truthfulness;
 - search;
 - clearance;
 - biometric gate where required.
-

Page 34 — Transport testing checklist

Test:

- routes;
 - vehicles;
 - drivers;
 - student assignment;
 - invoices;
 - maintenance;
 - fuel;
 - GPS seam;
 - tracker feed;
 - no fake connected GPS.
-

Page 35 — Cafeteria and inventory testing checklist

Test:

- stock items;
 - batches;
 - expiry;
 - low stock alerts;
 - meal models;
 - cards-only;
 - boarding-group meals;
 - no-cards mode;
 - sales and supplier relay.
-

Page 36 — Security testing checklist

Test:

- visitor sign-in;
- pickup verification;
- alternate pickup codes;
- panic alerts;
- passkeys;
- 2FA;
- impersonation logs;
- wrong-school gate;
- row-scoped access.

Audit logs are essential for support and safety.

Page 37 — Print quality rules

Print outputs should be document-first, not screenshot-first.

Rules:

- use A4 correctly;
- landscape for timetable where appropriate;
- portrait for class lists where appropriate;
- school logo top-left where useful;
- Powered by NEYO bottom-right or footer;
- avoid squeezed screen captures;
- content must fill the paper sensibly.

Future print audit should check every print route.

Page 38 — Deployment overview

Production should use:

- hosted Next.js app;
- Postgres/Neon database;
- object storage for files;
- proper environment variables;
- live Daraja credentials;
- SMS/email/WhatsApp credentials;
- observability;
- backups;
- custom domain.

Never deploy with dev mock credentials for real money flows.

See:

`docs/DEPLOY.md`

Page 39 — Environment variables to understand

Important categories:

- database URL;
- app base URL;
- session/security secrets;
- Daraja credentials;
- SMS provider credentials;
- email provider credentials;
- YouTube API key;
- object storage credentials;
- observability keys.

Founder rule:

Credentials are not public website content.

Page 40 — GitHub basics for founder

When files are changed, they appear as modified/untracked.

Typical workflow:

```
git status
git add .
git commit -m "Describe the completed feature"
git push
```

If many generated files appear, check `.gitignore` and avoid committing:

- node_modules;
- build outputs;
- cache folders;
- local secrets.

Ask for help before force-pushing.

Page 41 — Testing before commit

Before a serious commit:

```
./node_modules/.bin/prisma generate
./node_modules/.bin/prisma migrate deploy
NODE_OPTIONS=--max-old-space-size=1536 npm run typecheck
npm run test:roles
```

For feature-specific tests, run the script named after the feature, for example:

```
./node_modules/.bin/tsx scripts/i49-central-money-reconnect-test.ts
```

Page 42 — Screenshots rule

For every visual feature:

1. Run dev server.
2. Open the page.
3. Capture desktop screenshot.
4. Capture mobile screenshot when layout is important.
5. Save in `screenshots/`.
6. Update checklist/context anchor.

Example:

```
screenshots/i52-public-homepage-odoo-inspired-desktop.png
```

Page 43 — Marketing target: 100 schools by January

To reach 100 schools by January 2027, NEYO needs:

- a trusted public landing page;
- a demo approval flow;
- a school visit plan;
- referral loops;
- founder-led onboarding;
- WhatsApp follow-up;
- pricing clarity;
- simple printed pitch material;
- live product proof screenshots/videos.

The best early market is schools with pain in:

- fees;
 - receipts;
 - attendance;
 - report cards;
 - parent communication;
 - printing.
-

Page 44 — School visit blueprint

For each school visit:

1. Identify decision maker: Director, Principal, Bursar, ICT lead.
2. Ask pain questions.
3. Show only 3-5 relevant workflows.
4. Do not overwhelm them with every module.
5. Show fee receipt flow.
6. Show parent SMS/payment proof.
7. Show report card or ID card print.
8. Ask for pilot commitment.
9. Book onboarding date.
10. Follow up same day.

Best demo length: 18–25 minutes.

Page 45 — Sales conversation script

Opening:

“We built NEYO to help Kenyan schools run fees, attendance, exams, communication and documents in one place.”

Ask:

- How do you track fee balances today?
- How long does report card preparation take?
- Do parents get receipts instantly?
- Do teachers mark attendance digitally?
- What is the hardest office process each term?

Close:

“Can we set up your school and run one real workflow this week?”

Page 46 — What not to say in sales

Avoid:

- exaggerating;
- saying everything is fully automated if not;
- promising zero YouTube ads for third-party videos;
- promising hardware is connected before pairing;
- exposing internal architecture;
- using technical jargon;
- selling every module at once.

Say what is true, specific, and useful.

Page 47 — How NEYO helps Kenyan schools

NEYO helps schools by:

- reducing paper work;
- speeding fee follow-up;
- improving parent communication;
- making receipts traceable;
- organizing student records;
- improving attendance tracking;
- making reports cleaner;
- preserving data;
- reducing duplicated office work.

For parents, NEYO should feel simple.

For staff, NEYO should feel calm.

For owners, NEYO should feel controlled.

Page 48 — Scale confidence

NEYO can scale if the architecture is operated correctly:

- stateless app servers;
- Postgres with pooling;
- background jobs;
- object storage;
- CDN for static assets;
- queues for heavy tasks;
- observability;
- tenant indexes;
- careful permissions;
- no giant client bundles;
- no fake blocking integrations.

Scaling is not magic. It is discipline.

Page 49 — Founder weekly routine

Every week:

1. Check NEYO Ops metrics.
 2. Review customer threads.
 3. Review grace/suspended accounts.
 4. Review payments.
 5. Review landing page requests/waitlist.
 6. Review bugs and screenshots.
 7. Run role tests.
 8. Update checklist/context anchor.
 9. Pick one small feature batch.
 10. Talk to at least five schools.
-

Page 50 — Final founder operating rule

Never rush a feature because it “looks done.”

A NEYO feature is only complete when it has:

- real database state;
- validation;
- service logic;
- API endpoint;
- UI;
- loading/empty/error/populated states;
- Kenyan context;
- test or audit script;
- screenshot if visual;
- checklist update;
- context anchor update.

This manual is the starting operating guide. Keep expanding it as NEYO grows.

Appendix A — Core commands

```
npm install
./node_modules/.bin/prisma generate
./node_modules/.bin/prisma migrate deploy
npm run db:seed
NODE_OPTIONS=--max-old-space-size=1536 npm run typecheck
npm run test:roles
npm run dev
```

Appendix B — Key documents

docs/FEATURES-CHECKLIST.md
docs/CONTEXT-ANCHOR.md
docs/NEY0-BUSINESS-OS-ANALYSIS.md
docs/PAYMENTS-DEVELOPER-GUIDE.md
docs/YOUTUBE-MANAGEMENT-POSTING-STRATEGY.md
docs/MULTI-OS-READINESS.md
RUN-LOCALLY-FOR-FOUNDER.md

Appendix C — Current source of truth

The source of truth is always:

`docs/FEATURES-CHECKLIST.md`

The save game is:

`docs/CONTEXT-ANCHOR.md`

If those are not updated, the project memory becomes unreliable.